

OpenCourt

A free, open-source NCAA basketball analytics dashboard with ML-powered game predictions.

Kenneth Hinman · Project Manager / Backend

Alex Worthington · UI / UX Lead

Jackson Murphy · Data Pipeline / API

Samuel Bombrys · Analytics / ML

What We Built — and Why

What is OpenCourt?

A full-stack web app that pulls live NCAA Division I men's basketball data from the CBBData API, presents it through interactive dashboards, and uses a trained machine-learning model to predict the outcomes of upcoming games.

THE GAP WE FILL

Sites like EvanMiya and CBB Analytics paywall their advanced metrics. Academic prediction work lives in research papers and Kaggle notebooks. OpenCourt offers both — for free, in a real product.

Original Goals (from proposal)

- ✓ Ingest all 360+ D1 teams from a real sports API
- ✓ Browse teams, conferences, rankings, schedules
- ✓ Build a transparent ML prediction model
- ✓ Track prediction accuracy vs actual results
- ✓ Deploy a public, free, no-paywall site

Result: shipped to production at cs340-opencourt.up.railway.app

Architecture & Tech Stack



WHY THESE TECHNOLOGIES?

Backend	Django 6 + Python 3.14	Mature ORM, batteries-included admin, good fit for CRUD-heavy data app
Frontend	Tailwind v4 + DaisyUI v5	Utility CSS without writing CSS files; DaisyUI ships polished components
Charts	ApexCharts (CDN)	No build step, responsive, clean defaults for time-series stats
Database	SQLite (dev) / Postgres (prod)	Free local dev, free prod via Supabase, identical Django ORM in both
ML	scikit-learn + XGBoost	Tabular features, small dataset → tree ensembles outperform deep models
Hosting	Railway + Supabase + WhiteNoise	\$0 deploy, GitHub auto-deploys, no separate static-file CDN needed

Data Pipeline & Database

Two ingest modes

`manage.py sync_data`

Full backfill

~30+ API calls. Pulls all conferences → teams → season stats → games → game stats → rankings. Used for first-time setup or full rebuild. Idempotent via `update_or_create()`.

`manage.py update_data`

Incremental refresh

~5 API calls. Pulls only the last 30 days of games + current-season stats. Runs as a Railway cron job daily at 6 AM UTC to keep production fresh.

Reliability: exponential backoff on 429s, `Retry-After` header support, env-loaded API keys.

6 Django models

Conference	1 row per D1 conference
Team	~360 D1 programs (slug-routed)
TeamSeasonStats	Per-team, per-season aggregates
Game	Schedule + results, both teams
GameTeamStats	Per-game box scores (2 / game)
Ranking	AP / Coaches polls, weekly

FK chain: Conference → Team → TeamSeasonStats / Game → GameTeamStats

The Prediction Model

10-feature differential vector

Each feature is $\text{team_A} - \text{team_B}$ for the season.

<code>diff_efg_pct</code>	Effective FG% (3pt-weighted)
<code>diff_turnover_rate</code>	TO per possession
<code>diff_point_diff</code>	Avg margin per game
<code>diff_pace</code>	Possessions per 40 min
<code>diff_avg_ftr</code>	Free-throw rate
<code>diff_avg_rating</code>	Offensive rating
<code>diff_avg_blocks</code>	Blocks per game
<code>diff_avg_oreb_pct</code>	Offensive rebound %
<code>diff_avg_ato_rto</code>	Assist : turnover ratio
<code>home_advantage</code>	+1 home / 0 neutral / -1 away

Training pipeline

- 1 Pull 5 seasons of completed games from the DB
- 2 For each game, build the 10-feature diff vector
- 3 Label $y = 1$ if team A won, else 0
- 4 80 / 20 stratified train-test split
- 5 Train RandomForest + XGBoost classifiers
- 6 Pickle best model $\rightarrow$ models/RandomForest_model
- 7 Django views load it once per worker (cached)

Why trees? Small tabular dataset $\rightarrow$ ensembles beat deep nets.

Model Performance

71.44%

Accuracy on the 2025–26 season

5,752 completed games predicted · 4,109 correct · 0 skipped

HOW IT COMPARES

Always-pick-home baseline



Published ML literature



OpenCourt RandomForest



OpenCourt XGBoost (alt)



Strongest when: clear stat mismatch. **Weakest when:** upsets, injuries, late-season streaks the box score doesn't capture.

UI & Frontend

Design decisions

Tailwind v4

Utility-first CSS — no custom stylesheets. JIT-compiled.

DaisyUI v5

Pre-styled cards, tabs, badges. Consistent look without designing components from scratch.

ApexCharts

Drop-in interactive charts via CDN. Handles team detail page time-series.

Class-based views

7 Django CBVs. Server-rendered HTML; AJAX endpoint for live search.

Mobile responsive

Tested at 390px, 768px, 1280px breakpoints.

7 pages, 1 codebase

/	Home — top teams, featured conferences
/teams/	Searchable list of ~360 D1 teams
/conferences/	Sidebar + aggregate conference stats
/rankings/	AP & Coaches Poll side-by-side
/upcoming/	Predictions + completed-game results
/<team-slug>/	Team detail w/ ApexCharts + box scores
/about/	Team + project info

Deployment & Hosting

RAILWAY

Web Service

gunicorn + Django · WhiteNoise for static files · Auto-deploys on merge to main

Cron Service

python manage.py update_data · Daily at 06:00 UTC · Keeps stats fresh

\$0/mo Railway \$5 trial + Supabase free tier

SUPABASE

Managed PostgreSQL

- 500 MB free tier — well under our usage
- Transaction-pooler mode (port 6543)
- Same Postgres for web + cron
- **DATABASE_URL** injected via env vars

GitHub push → live in ~2 min *(no manual steps)*

LIVE DEMO

Let's see it.



<https://cs340-opencourt.up.railway.app/>

THANK YOU

Questions?

Visit: cs340-opencourt.up.railway.app

Repo: github.com/cs340-26/OpenCourt

Kenneth Hinman · Alex Worthington · Jackson Murphy · Samuel Bombrys

CS 340 · University of Tennessee, Knoxville · Spring 2026

Did We Meet Our Goals?

FROM THE PROPOSAL

- ✓ Free, no-paywall public site
- ✓ All 360+ D1 teams ingested
- ✓ ML predictions w/ accuracy tracking
- ✓ Conference & ranking views
- ✓ ApexCharts visualizations
- ~ User accounts + saved teams
- ~ Stored predictions in DB

shipped
shipped
shipped
shipped
shipped
scrapped
future

WHAT'S NEXT

- 1 **Persist predictions to the DB**
Stop recomputing on every page-load. Enables season-wide accuracy stats with a single query.
- 2 **Strength of schedule + injuries**
Richer features the box score doesn't capture.
- 3 **Confidence scores + ensembling**
Soft-vote RF + XGBoost; expose probability to the UI.
- 4 **User accounts**
Save favorite teams, alert on upset predictions.
- 5 **Bracket builder**
Plug predictions into a March Madness bracket generator.

Challenges & How We Solved Them

PERFORMANCE

Predictions on every game = timeouts

Problem: Results page tried to run the ML model against ~5,000 games per request. Gunicorn workers timed out.

Fix: Paginate the DB query first (50 games / page), then enrich. Cache the loaded model at the worker level.

FRONTEND

Tailwind v4 migration

Problem: Tailwind v4 ships a new @source directive — classes weren't compiling because templates weren't on the source path.

Fix: Explicit @source declaration in the theme config. Locked to Node ≥ 18 in the README.

DATA QUALITY

Null stats → ZeroDivisionError

Problem: Some teams missing season stats early in the year broke the feature builder when computing TO ratios.

Fix: Defensive guards on every divisor; predictions return early with a 'no prediction' badge instead of 500-ing.

DEPLOYMENT

Local SQLite ≠ Prod Postgres

Problem: A few queries that worked in SQLite hit case-sensitivity / null ordering differences in Postgres.

Fix: Same Django ORM in both environments; explicit ordering and Lower() on case-sensitive comparisons.

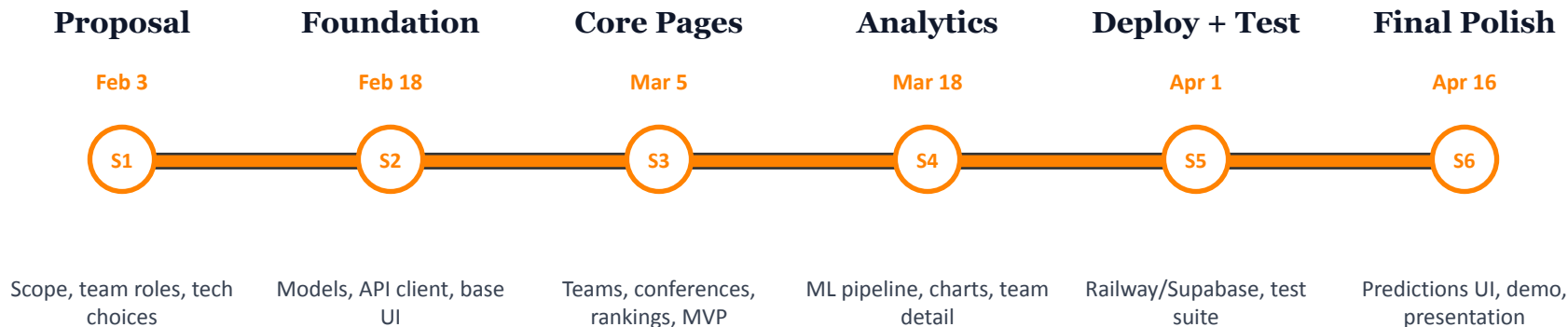
Testing & Code Quality

WHAT WE TEST

<code>test_models.py</code>	37 tests	Field validation, properties, FK relationships, slug auto-gen
<code>test_views.py</code>	41 tests	Integration tests for all 7 CBVs, status codes, context data
<code>test_services.py</code>	48 tests	Mocked CBBDData responses, retry/backoff, idempotency
<code>test_predictions.py</code>	16 tests	Feature builder, model loading, prediction shape
<code>test_stats.py</code>	13 tests	Helper math: win %, point diff, edge cases (0–0)
<code>test_urls.py</code>	16 tests	Routes resolve, slug params accepted, namespacing

SPRINT TIMELINE

Six Sprints, One Semester



Working agreement: two-week sprints, GitHub Issues + project board, weekly stand-ups, status report each sprint, PR reviews required before merge to main.